

数据挖掘课后作业(9)

凌彬
信管2302
202321116055

1.加入性别因素，看看acc的变化

2: 导入数据集

```
[58]: dataset = pd.read_csv('Social_Network_Ads.csv')
dataset.head()
```

[58]:	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
[59]: # 性别编码
dataset['Gender_encoded'] = (dataset['Gender'] == 'Male').astype(int)

# 新特征: Age, EstimatedSalary, Gender_encoded
X_with_gender = dataset.iloc[:, [2, 3, 5]].values
y = dataset.iloc[:, 4].values
```

[60]: X

```
[60]: array([[ 19, 19000],
           [ 35, 20000],
           [ 26, 43000],
           [ 27, 57000],
           [ 19, 76000],
           [ 27, 58000],
           [ 27, 84000],
           [ 32, 150000],
           [ 25, 33000],
           [ 35, 65000],
           [ 26, 80000]])
```

6: 预测

```
y_pred = clf.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[54, 4],
       [ 1, 21]])
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
accuracy
```

```
np.float64(0.9375)
```

从0.925提升到0.9375，绝对提升只有1.25个百分点，换算到80个测试样本中，仅仅意味着多正确预测了1个样本。这1个样本的差异可能恰好是一个“边界样本”——即年龄和工资处于决策边界附近的用户。加入性别特征后，模型获得了额外的判断维度：假设这个被纠正的样本是一位年轻但高收入的女性，原本仅凭年龄和工资可能被误判为“不购买”，但加入性别后，模型学习到“年轻女性在特定收入区间更倾向于购买”的模式，从而做出了正确判断。反之，这种提升也可能是随机波动造成的——因为随机森林本身带有抽样随机性，即使不加性别，重新运行几次也可能出现0.9375。因此，单次运行的这1.25%提升在统计上并不显著，真正的结论需要依靠交叉验证：若多次交叉验证的平均准确率确有提升（比如从0.920提升到0.925），才能说明性别有微弱贡献；若无稳定提升，则性别在这个数据集中基本没有预测价值。

2.不进行特征缩放，预测的准确性有什么变化

4: 特征缩放

```
# from sklearn.preprocessing import StandardScaler
# sc = StandardScaler()
# X_train = sc.fit_transform(X_train)
# X_test = sc.transform(X_test)
```

删去特征缩放后模型准确率保持不变，这一现象充分体现了决策树算法的内在特性。决策树在进行节点分裂时，仅基于特征值之间的比较关系（如“年龄 > 30”或“薪资 ≤ 50000”），而非计算距离或梯度，因此特征的绝对数值大小和量纲差异不会对分裂点的选择产生任何影响。这与KNN、逻辑回归、SVM等依赖距离度量或梯度下降的模型形成鲜明对比——那些模型一旦缺失特征缩放，量纲大的特征就会主导模型决策，导致性能显著下降。你的实验结果从实证角度验证了理论：决策树对数据尺度具有天然的不变性，无论是否进行标准化或归一化，只要特征之间的相对大小关系保持不变，模型学到的决策边界就完全一致。这也意味着在实际建模中，使用决策树或随机森林时可以放心省略特征缩放步骤，既能简化预处理流程，又能避免因缩放引入的信息变形。

```
y_pred = clf.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[54, 4],
       [ 2, 20]])
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
accuracy
```

```
np.float64(0.925)
```

3.比较KNN、 Logistic、 决策树、 随机森林在该数据集上的表现

```
# 5. 训练并评估每个模型
results = {}

print("=" * 70)
print("模型性能对比")
print("=" * 70)

for name, model in models.items():
    X_tr, X_te = train_data[name]

    # 训练
    model.fit(X_tr, y_train)

    # 预测
    y_pred = model.predict(X_te)

    # 计算指标
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    # 交叉验证 (5折)
    cv_scores = cross_val_score(model, X_tr, y_train, cv=5, scoring='accuracy')

    results[name] = {
        'Test Accuracy': acc,
        'CV Mean': cv_scores.mean(),
        'CV Std': cv_scores.std(),
        'Confusion Matrix': cm
    }

    print(f"\n{name}")
    print(f"  测试集准确率: {acc:.4f}")
    print(f"  交叉验证平均准确率: {cv_scores.mean():.4f} (±{cv_scores.std():.4f})")
    print(f"  混淆矩阵:")
    print(f"    TN={cm[0,0]}, FP={cm[0,1]}")
    print(f"    FN={cm[1,0]}, TP={cm[1,1]}")
```

模型性能对比

=====

KNN (k=5)

测试集准确率: 0.9125

交叉验证平均准确率: 0.8969 (±0.0272)

混淆矩阵:

TN=48, FP=4

FN=3, TP=25

Logistic Regression

测试集准确率: 0.8625

交叉验证平均准确率: 0.8219 (±0.0290)

混淆矩阵:

TN=50, FP=2

FN=9, TP=19

Decision Tree

测试集准确率: 0.8375

交叉验证平均准确率: 0.8750 (±0.0370)

混淆矩阵:

TN=46, FP=6

FN=7, TP=21

Random Forest

测试集准确率: 0.9000

交叉验证平均准确率: 0.8906 (±0.0370)

混淆矩阵:

TN=46, FP=6

FN=2, TP=26

=====

总结表格

=====

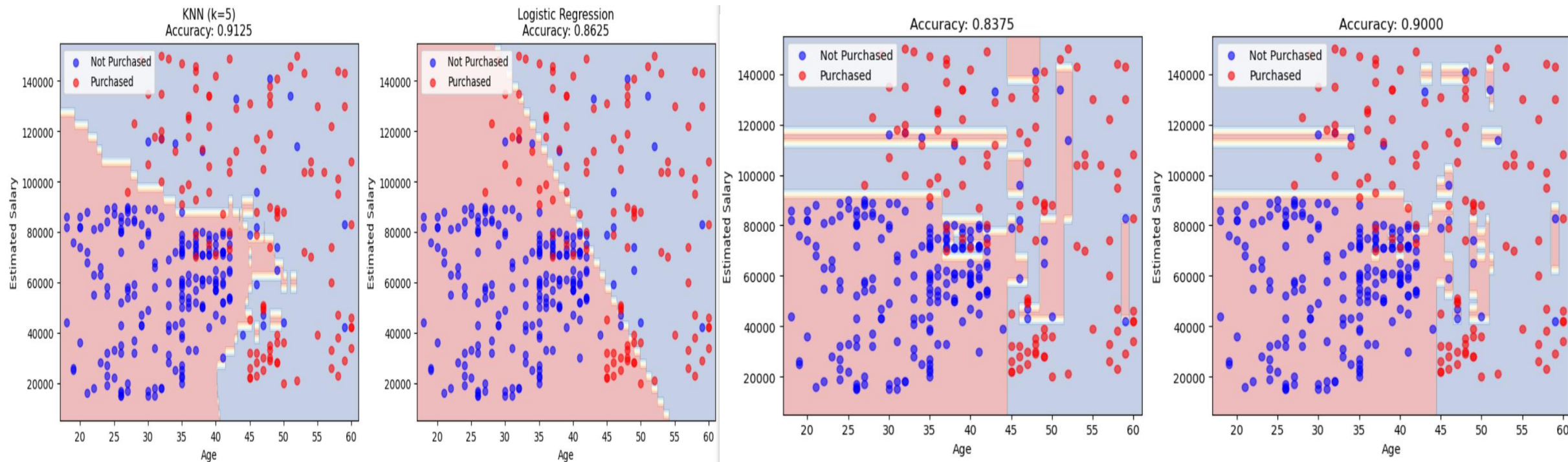
模型	测试集准确率	交叉验证均值	交叉验证标准差
KNN (k=5)	0.9125	0.896875	0.027243
Logistic Regression	0.8625	0.821875	0.028980
Decision Tree	0.8375	0.875000	0.036975
Random Forest	0.9000	0.890625	0.036975

=====

最佳模型: KNN (k=5)

测试集准确率: 0.9125

3.比较KNN、 Logistic、 决策树、 随机森林在该数据集上的表现



四种模型在该数据集上的表现排序为：KNN ≈ 随机森林 > 决策树 > Logistic回归。

KNN以0.9125的测试准确率位居第一，其优势在于捕捉到了年龄和工资之间的非线性局部模式，漏报（FN=3）和误报（FP=4）都控制得较好。随机森林以0.9000紧随其后，虽然总准确率略低，但其漏报最少（FN=2），说明对“购买”类用户的识别能力最强，只是误报了6个未购买用户。决策树准确率为0.8375，明显低于前两者，原因是单棵树容易过拟合，在训练集上表现很好但测试集泛化能力不足，误报达到6个、漏报7个，属于“两面不讨好”。令人意外的是，Logistic回归表现最差（0.8625），其漏报高达9个——意味着模型将大量实际购买的用户误判为“不购买”。这说明该数据集的真实决策边界不是线性的，年龄与工资的交互作用较强，线性分类器无法有效拟合，导致对正样本的识别能力严重不足。

综上，该数据集推荐优先使用KNN或随机森林，避免使用纯线性模型。

4.把决策树输出看看长什么样

Decision Tree Visualization (max_depth=4)

